AMITY UNIVERSITY
Kolkata

# Report : ParkSure

Live prototype: prateekcandwill.github.io/parksure-smart-parking-kolkata
Source code: github.com/Prateekcandwill/parksure-smart-parking-kolkata


**Name of the Student** : Prateek Kumar
Pritam Bhakta
Rajasree Dutta
Mittika Jana

**Enrolment No** : A910119824003
A910119824029
A910119824023
A910119824031


**Program** : B.Tech AI
**Semester** : V
**Batch** : 2024 - 2028
**Course Name** : Design Thinking for Innovation
**Course Code** : IPDE211

**Name of the Faculty Coordinator :** Dr Saberi Goswami

# 1. Introduction and problem statement

## 1.1 Background

Parking in Kolkata is a guessing game. A driver circling Park Street does not know whether a bay is free, what it will cost, or whether the barrier will open on arrival. While they guess, they add to traffic on roads that are already full.

We picked four places that handle cars in very different ways. Park Street Centre is a multistorey lot in the middle of the city. Salt Lake Sector V is an office district with a sharp weekday peak. Howrah Station Forecourt has surface bays that turn over fast. The Maidan near Victoria Memorial is an open ground where bays cannot be booked one by one. One system had to work for all four.

## 1.2 Why design thinking

Parking looks like a technology problem, but the trouble starts with people. The driver, the attendant and the operator each see only one side of the same lot. Design thinking made us start with them before choosing any technology. We followed the five usual stages. In Empathize we studied a hurried driver and a pressured attendant. In Define we turned that into a point of view and design principles. In Ideate we brainstormed and filtered ideas. In Prototype we drew wireframes and then built a working React app. In Test we ran automated checks and a manual test script.

## 1.3 Problem statement

We found three problems on the street. Drivers cannot tell whether a lot has space before they arrive, so they circle and add to congestion. A price that rises with demand feels unfair unless the driver can see why it changed. And drivers and staff use different tools, so the gate, the cash and the records often disagree.

*Point of view: A driver near Park Street needs a trustworthy, live answer to "is there a bay, and what will it cost?", and the people running the lot need the same answer on their side of the barrier.*

Our objectives followed from this. The prototype had to show live bay estimates, explain every price, connect driver and staff in one flow, and work one-handed on a phone in English, Bengali and Hindi.

# 2. Research and target group analysis

## 2.1 Approach

Our empathy work centred on two users who feel opposite pressures. The driver is in a hurry and wants certainty. The attendant has a queue behind the barrier and wants one clear thing to do next. We used empathy mapping, personas and journey mapping to turn what we learned into design requirements.

## 2.2 Target groups

The driver is the main user. Drivers want to park quickly, close to where they are going, at a fair price, and their pain is not knowing whether space exists or what it will cost. The attendant wants to let the right cars in and out without delay, but deals with queues, plate mismatches and cash. The operator keeps the whole lot running and needs sensor faults, alerts and shift cash in one place. The administrator sets prices and rules, and needs every change to leave a record.

### 2.3 Personas

Aritra is an office commuter who drives a petrol sedan into central Kolkata most days. He wants a bay before he reaches the area, and he distrusts any price that changes without a reason. He uses his phone with one hand, at a signal or in slow traffic.

The attendant works long shifts at the barrier with a queue behind every car. Gloves or wet hands make small buttons hard to hit. He has to check plates, handle cash and sort out exceptions, so he needs a screen that tells him what to do next.

### 2.4 Empathy map of the driver

Aritra says, "Is there any space?" and "Why is it costlier now?" He thinks about being late and whether the gate will open for him. He circles the block, calls the lot or asks around. He feels anxious about time and suspicious about price.

### 2.5 What we learned

- Uncertainty costs more than the fee. Circling is what hurts.
- A surge price is accepted only when the driver sees the reason for it.
- A dense screen at a busy gate causes mistakes, so staff need one decision at a time.
- Driver and staff need the same answer, which led to the four-role model on one account.
- The app must work with one hand, with large touch targets, in English, Bengali and Hindi.

## 3. Idea generation and brainstorming

### 3.1 How might we questions

We brainstormed around four questions and held back judgement until the lists were long.

- How might we tell a driver a bay is free before they arrive?
- How might we make a changing price feel fair?
- How might we let an attendant act with one tap and no doubt?
- How might we keep the gate, the cash and the app telling the same story?

### 3.2 What we kept and what we dropped

We then grouped the ideas and filtered them by value to the user and effort to build. We kept live bay estimates with a confidence score, a demand-based price with a visible breakdown, and a camera-based estimate for the Maidan, where bays cannot be booked. We kept the four roles on one account, a shared audit trail with a reason for every change, and a QR pass that opens the gate. Slide to pay and the scratch reveal stayed too, with button and keyboard fallbacks.

Three ideas were dropped. An open-ended AI chatbot would need a server and could leak data, so we built a rule-based assistant that works offline and keeps everything on the device. Google Maps inside the app needs an API key and billing, so we used OpenStreetMap. Built-in turn-by-turn navigation was too heavy, so the app hands the route to the driver's own maps app.

### 3.3 Design principles

- Show the reason behind every number.
- Give staff one decision per screen.
- Say plainly that the demo is simulated.
- Design for one hand on a Kolkata street.
- Keep the demo self-contained so anyone can open it.

# 4. Sketches, visualisation and prototype

## 4.1 Low-fidelity wireframes

We drew black and white wireframes first, so the team argued about layout and flow and not about colour. The set covers five mobile screens at 390 by 844 pixels: Home, Search and Filter, Place Detail, Profile and Settings. Place Detail puts the price, the live bay grid and a single main action (Book a bay) on one screen. Search and Filter offers chips such as Free now and Covered, and a list or map toggle.

## 4.2 Visual design

The final look uses a near-black canvas with a gold accent for action. The operator workspace switches to violet so it feels distinct. Headings use Space Grotesk and body text uses Inter. Plates and amounts use JetBrains Mono so digits cannot be confused at a gate. Cards have large rounded corners, touch targets are 48 pixels (72 pixels for the attendant), and motion appears only to confirm an action, with reduced motion respected. Status never depends on colour alone, since every badge also carries text and a shape.

## 4.3 High-fidelity prototype

We turned the wireframes into a working React app with real Kolkata place names, prices and number plate formats. Figure 2 shows four screens from the live demo.

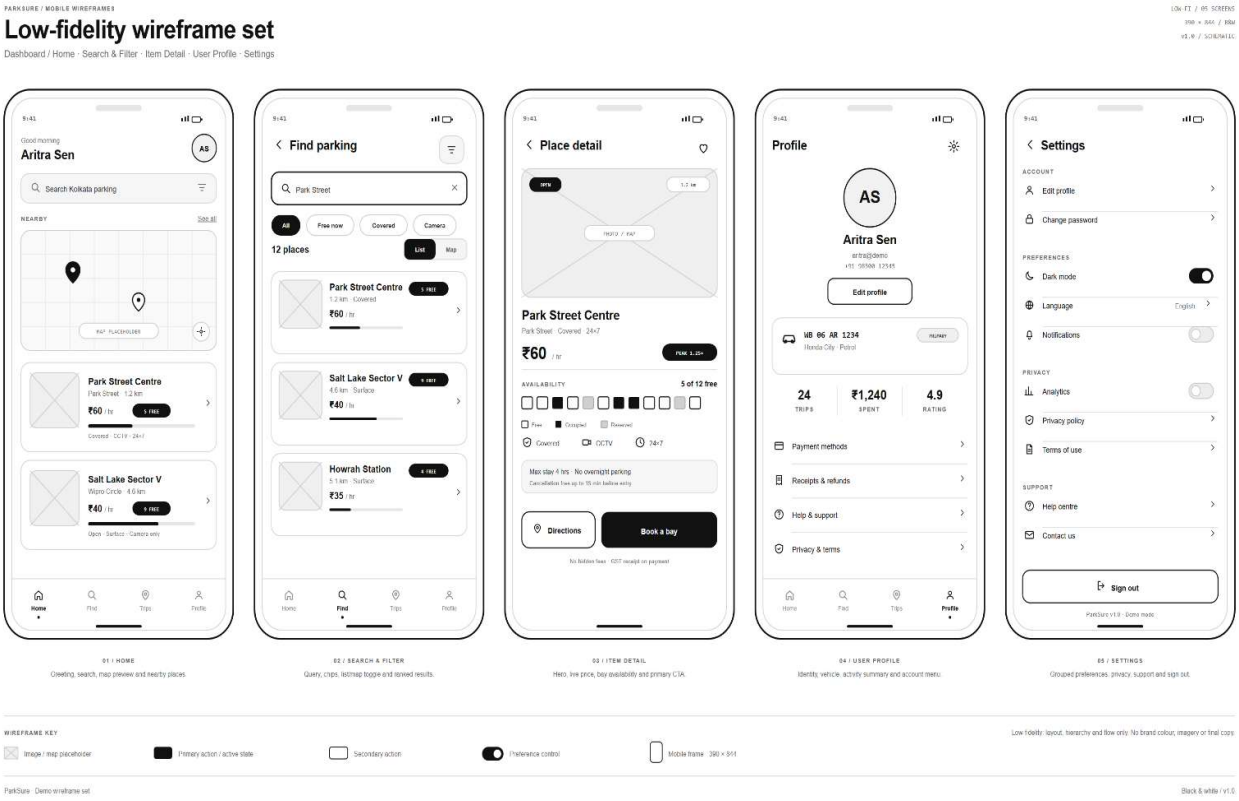
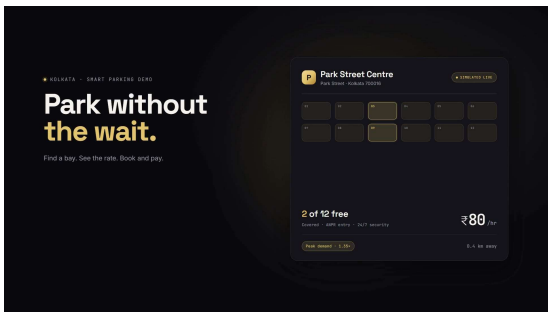
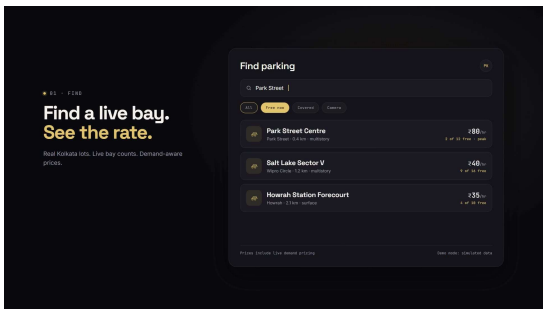


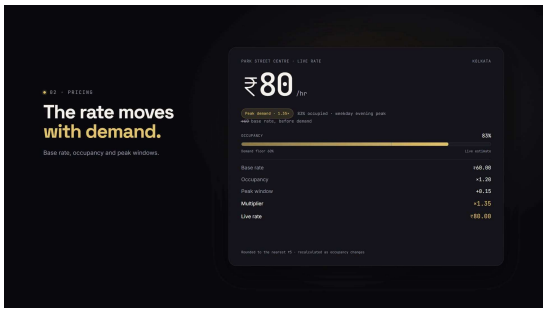

*Figure 1: Low-fidelity wireframe set from the project repository*



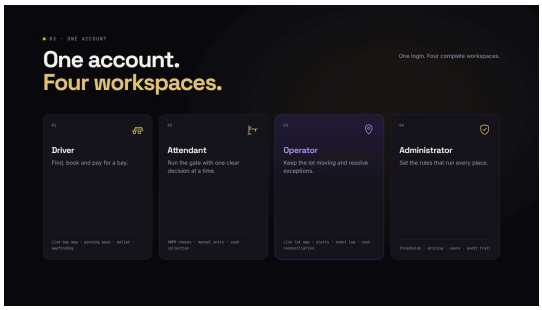

*Home and live bay card*




*Find and filter*




*Dynamic pricing breakdown*




*Four workspaces*

*Figure 2: Final screens of the ParkSure prototype*

### 4.4 Testing

We checked 38 colour pairs against the WCAG AA contrast rules, including the light theme. We loaded all 28 routes at 390 and 320 pixel widths to catch sideways scrolling, and we checked every internal link. A smoke test and a manual script in the test checklist cover the main flows. We also wrote down the gaps we found: the OTP field can lose its middle digit when edited in some browsers, and map pins respond best to a pointer, so the list view is the accessible fallback.

## 5. Proposed solution and implementation

### 5.1 The solution

ParkSure is a mobile-first web app. It installs as an offline PWA and also ships as an Android app built with Capacitor. One login opens four workspaces, so a single parking session can be followed from search to exit.

The driver workspace shows live bay estimates with a confidence score, the price and its reason, a map and directions, booking, a QR pass, payment by card, UPI or wallet, a GST receipt as PDF, refunds and trip history. The attendant workspace handles plate checks, manual entry and cash collection, one decision at a time. The operator sees a live lot map, alerts, an event log and end-of-shift cash reconciliation. The administrator sets occupancy and price thresholds, manages user roles, and reads an audit trail that exports to CSV.

### 5.2 Explainable dynamic pricing

The price is never a hidden number, and the app shows the arithmetic like a receipt. Take Park Street Centre on a weekday evening. The administrator sets a base rate of Rs 60 per hour. The lot is 83% full, above the 60% demand floor, so the occupancy lift is 1.20. The 5 pm to 9 pm weekday peak adds 0.15, which gives a multiplier of 1.35. Rs 60 times 1.35 is Rs 81, and rounding to the nearest Rs 5 gives a live rate of Rs 80 per hour.

The multiplier is kept between 0.85 and 1.60, and a night discount applies from 10 pm to 6 am. The base rate appears struck through beside the live rate, so the driver can always see why the price moved.

### 5.3 Driver journey

The driver searches and filters, sorting by nearest, most free or cheapest, in list or map view. The place page shows the live bay map, the price with its reason and one directions button. Booking is a single screen where the driver picks a duration and vehicle, reads the price breakdown and slides to pay. The bay number is then revealed under a scratch card, with a Reveal button as a fallback, and the driver gets a live QR pass with a countdown and options to extend or exit.

### 5.4 Implementation

The interface uses React 19, TypeScript, Vite, Tailwind CSS and Framer Motion. State lives in Zustand, and a Zod-checked local API sits over browser storage. Maps use Leaflet with OpenStreetMap data and CARTO tiles. Receipts are made with jsPDF, passes use QR codes, and the PWA layer uses Workbox.

There is no server. The "API" is a typed module over browser storage, so it can be swapped for real HTTP services later without changing the screens. Payments, sensors, plate reads and gate openings are all simulated, and the app says so in plain words.

### 5.5 Limits and next steps

- Move data, login and payment approval to a real server and database.
- Replace the demo audit hash with SHA-256 and server-side keys.
- Connect real camera, plate reader and sensor feeds to the existing bay confidence model.
- Complete the Bengali and Hindi text for long screens and add end-to-end tests.

AI tools helped us write and refine code, screen text and documentation. The team defined the problem, point of view, four-role concept and design principles, and reviewed and tested the output.

## 6. Conclusion and references

### 6.1 Conclusion

The parking problem turned out to be about trust more than features. Starting from two real pressures, a hurried driver and a busy attendant, gave us one point of view: both sides need the same live answer. That idea produced the four-role model, the visible price receipt, the one-decision attendant screen and the audit trail.

Wireframing before styling saved time, because arguments stayed on flow. Saying no to the chatbot, Google Maps and built-in navigation kept the prototype small enough to finish and open for anyone to run. The weak point is that all data is simulated and no real drivers have tested it. The next step is a trial at one of the four sites with real sensors, real payments and drivers who did not build the app.

### 6.2 References


[1] Brown, T. (2008). Design Thinking. Harvard Business Review, June 2008.

[2] Hasso Plattner Institute of Design at Stanford (d.school). An Introduction to Design Thinking Process Guide.

[3] World Wide Web Consortium (W3C). Web Content Accessibility Guidelines (WCAG) 2.1.

[4] OpenStreetMap contributors. Map data, openstreetmap.org/copyright. Tiles by CARTO.

[5] Leaflet, React, Vite, Capacitor and Zod project documentation.